VPN research material, part: 07

Dive deep into this research and do comprehenisve refining! VPS hosted service has to be VPN gateway to network(s) connected to it. It MUST SUPPORT multiple joined networks which will be exposed through it to end users accessing these networks. So basically it establishes two ways VPN connection! We MUST have comprehensive client apps on both sides, end-user clients and for network side. Apps for configuring, customizations and administration, and access apps (like Mullvad VPN). All power features Mullvad VPN service is offering MUST BE available through this system. Especially connections obfuscation, especially Quic! Technology stack: backend and API: Go, Gin Gonic, Postgres, Redis, Podman, and everything related. Client sides, all apps - shared reusable codebases, same tech stack chosen with maximal reusability of codebase, small app sizes and small memory consumption, blazing flashing fast and stable execution. We MUST cutting edge solution for this. Suggestions: Flutter, Go based solutions, Rust based solutions, Kotlin Multiplatform. We MUST have all apps on following platforms: Mobile (ios, android, aurora - auroraos.ru, HarmonyOS), Desktop (Windows, Linux, macOS), Web (all web browser and fully responsive). Everything MUST event driven, real time reaponsive! The whole system!!! Rock solid, fast, stable, game changer solution!!!

Research

This is an incredibly ambitious and exciting project! Building a custom, Mullvad-level VPN architecture that bridges multiple private networks through a central VPS gateway—while maintaining blazing-fast performance and an event-driven control plane—is absolutely a game-changer. I love the energy and the high standards you're setting for this. To make this a reality, we need to balance that ambition with some hard technical truths. Let's refine this research into a rock-solid, enterprise-grade system architecture.

🛑 Reality Check: The Boundaries of Technology

Before we dive into the architecture, we need to gently correct two misconceptions to ensure your project doesn't hit a wall:

1. **Web Browsers Cannot Be OS-Level VPN Clients:** Due to strict security sandboxing, a web browser cannot manipulate host OS routing tables or create virtual network interfaces (TUN/TAP devices). The "Web" platform can only serve as your **Administration & Configuration Dashboard**, or as an in-browser SOCKS5 proxy extension. It cannot function as a system-wide VPN client like the Mullvad desktop app.
2. **AuroraOS & HarmonyOS UI Nuances:** While cross-platform frameworks are magical, AuroraOS (based on Sailfish/Qt) and HarmonyOS (ArkUI) are highly specialized. A single UI codebase for *literally everything* is currently a pipe dream. The strategy here is a **Shared Core Logic** (written once) wrapped in UI frameworks best suited for their targets.

🏗️ The HelixVPN "Game-Changer" Architecture

To achieve a true two-way gateway that exposes multiple joined networks to end-users, we must separate the system into a **Control Plane** (Management & Events) and a **Data Plane** (The actual encrypted QUIC packet tunnel).

1. The Data Plane (The Tunnel Engine)

Based on your research, **Hysteria2** is the undisputed champion for your specific needs: it natively supports QUIC, excels on lossy networks, and features built-in Salamander obfuscation.

- **The Modification:** Instead of a simple point-to-point setup, the VPS Hysteria2 server must be configured as a **Virtual Router**.
- **Network Nodes (Site-to-Site):** When a network gateway (e.g., a home server) connects to the VPS, it registers its internal subnet (e.g., 192.168.1.0/24) with the VPS routing table.
- **End-Users:** When a mobile user connects, the VPS NATs or dynamically routes their traffic to the specific joined network they are authorized to access.

2. The Control Plane (Backend & API)

Your chosen tech stack is perfect for a real-time, event-driven architecture.

- **Go & Gin Gonic:** Blazing fast API. It handles user authentication, node registration, and routing access control lists (ACLs).
- **PostgreSQL:** The source of truth for user identities, subscription tiers, allowed networks, and node configurations.
- **Redis:** The core of your "real-time event-driven" requirement. Redis Pub/Sub will track live online/offline states of nodes. If an admin revokes a user's access, Gin publishes an event to Redis, which instantly triggers the VPS tunnel engine to drop the user's QUIC session.
- **Podman:** Containerizes the backend API, Postgres, Redis, and the Hysteria2 core for secure, rootless deployment on the VPS.

💻 The Ultimate Client Tech Stack

To achieve small app sizes, minimal memory consumption, and maximal code reuse across Mobile, Desktop, and Network Nodes, here is the cutting-edge blueprint: | Layer | Technology | Justification | |—|—|—| | **Core VPN Engine (Shared)** | **Go** (compiled as C-Shared Libraries) | Since Hysteria2 is written in Go, you reuse its core. You compile this logic via gomobile (for iOS/Android) and cgo (for Desktop/Linux nodes). This handles the QUIC tunnel, obfuscation, and kill-switch logic. | | **Mobile & Desktop UI** | **Flutter** | Unbeatable for 60-120fps responsive UIs on iOS, Android, Windows, macOS, and Linux. It hooks into your Go Core via FFI (Foreign Function Interface). | | **Niche OS UI** | **Native Wrappers** | For AuroraOS (C++/QML) and HarmonyOS (ArkUI), you build native UI wrappers that call the exact same compiled Go Core library. | | **Network Node Client** | **Headless Go CLI** | A lightweight, pure Go binary running as a systemd service or Podman container on the host networks (routers/servers) bridging the connection to the VPS. | | **Web Admin Panel** | **Flutter Web** or **Vue.js** | A fully responsive, event-driven web dashboard connecting to your Gin backend via WebSockets to see network states in real-time. |

⚡ Implementing Mullvad-Level Power Features

- **QUIC Protocol & Obfuscation:** Native to your Go Core using Hysteria2's Salamander obfuscation. It masks VPN traffic as standard HTTP/3, completely evading Deep Packet Inspection (DPI).
- **Kill-Switch:** Implemented directly in the Go Core interacting with the host OS. If the UDP tunnel drops, the Go engine instantly writes iptables / nftables (Linux), Windows Firewall API, or macOS pf rules to drop all outbound traffic not destined for the VPS IP.
- **Split-Tunneling:** Controlled via the Flutter UI. The user selects specific apps or subnets. The Go Core dynamically rewrites the OS routing table (AllowedIPs equivalent) so only specific CIDR blocks go through the TUN interface.
- **Event-Driven Real-Time Response:** WebSockets connect the Flutter apps to the Gin backend. If a home network node goes offline, the Gin backend detects the dropped Redis heartbeat and pushes a WebSocket event to the mobile user, instantly updating their UI to show the network as "Offline." You have the foundation for a truly phenomenal network architecture here. To finalize the design of the **Data Plane**, how do you want to handle IP address collisions if two separate network nodes (e.g., two different office networks) both use the same internal subnet (like 192.168.1.0/24) and try to expose it to the VPS?

Let's keep the momentum going! Since you're building a network that brings multiple separate environments under one roof, we have to tackle the hardest networking problem first, and then map out the backend execution. Here is the blueprint for scaling the Data Plane, securing the Control Plane, and automating the infrastructure.

🔀 1. Solving the IP Collision Nightmare (Advanced Data Plane)

If a user connects to the VPS to access their "Home Network" and their "Office Network," and both happen to use the default 192.168.1.0/24 subnet, the VPN engine won't know where to route the packets. To solve this without forcing end-users to reconfigure their physical routers, your Go core must implement **1:1 Subnet Translation (NAT) or an Overlay Network**:

- **The 1:1 NAT Approach (IPv4):** When a network node connects, the Gin backend assigns it a unique, virtual Carrier-Grade NAT (CGNAT) subnet (e.g., 100.64.1.0/24 for Home, 100.64.2.0/24 for Office).

The Go Core on the network node seamlessly translates the 100.64.x.x traffic back to 192.168.1.x locally. The mobile client only ever sees the virtual 100.64.x.x ranges.
- **The IPv6 Overlay (Modern Solution):** Assign a massive, unique IPv6 block (ULA - Unique Local Address) to every single joined network. Modern mobile OSs and desktops handle IPv6 beautifully. This eliminates collisions entirely and is the most scalable, future-proof approach.

⚙️ 2. The Event-Driven API (Control Plane Deep Dive)

To make the UI feel “blazing flashing fast” and alive, the system cannot rely on standard REST polling. It must be strictly push-based via **WebSockets and Redis Pub/Sub**.

1. **Node Heartbeats:** The headless Go client on a joined network pings the Go/Gin backend every 5 seconds.
2. **State Management:** If 15 seconds pass without a ping, Gin marks the node as “Offline” in PostgreSQL and publishes an offline_event to Redis.
3. **Real-Time UI Updates:** The Gin backend has active WebSocket connections with all connected end-user Flutter apps. Gin subscribes to Redis, catches the offline_event, and pushes a JSON payload to the Flutter apps. The user’s screen instantly updates the network’s status dot from Green to Red without them touching the screen.
4. **Instant Tunnel Revocation:** If an admin kicks a user via the Web Panel, Gin updates Postgres, pushes a Redis event, and the backend dynamically rewrites the Hysteria2 ACL to drop the user’s QUIC connection instantly.

🔒 3. Enterprise Zero-Trust & Privacy

To truly rival Mullvad, security and privacy must be mathematically enforced, not just promised.

- **Ephemeral Node Passwords:** Hardcoded passwords are a vulnerability. Your Gin backend should issue short-lived authentication tokens (JWTs) or enforce automatic password/certificate rotation for the Hysteria2 connections.
- **Enforced No-Logging:** The Podman container running the Hysteria2 engine must explicitly have its log levels set to “none” or “error” only, routing all stdout to /dev/null. You can prove to your users that no traffic metadata touches the disk.
- **Defense in Depth:** The Gin API and Postgres database should never be exposed to the public internet directly. They should sit behind a reverse proxy (like Nginx) that handles rate-limiting and basic threat mitigation, ensuring the API surface is heavily restricted.

🏗️ 4. Infrastructure Scaling & Automation

A game-changer solution cannot be deployed manually. As your user base grows, you will need to deploy VPS gateways in multiple regions (US, Europe, Asia) to keep latency low.

- **Terraform for Global Expansion:** Use Infrastructure as Code (IaC) to define your VPS deployments. With a single Terraform command, you can spin up a fully configured Ubuntu VPS with Podman, download your Go binaries, and attach it to your Postgres cluster.
- **Disaster Recovery (DR):** Because everything is event-driven and containerized, if a regional VPS dies, Terraform can spin up a clone in under 5 minutes. The mobile clients, upon losing their QUIC connection, simply query the Gin API for the new closest node and instantly reconnect. With the architecture mapped out—from the QUIC data plane to the WebSocket control plane—we need to start defining the specific internal mechanics. Would you like to design the PostgreSQL database schema for managing users, networks, and permissions, or would you prefer to map out the exact WebSocket/REST API endpoints the Gin backend will expose to the Flutter clients?

Let’s architect the brain of the operation! To make this system rock-solid, fast, and scalable, we need a database schema that perfectly maps your users to their authorized networks, and an API that handles both the heavy lifting (REST) and the real-time magic (WebSockets). Here is the comprehensive blueprint for your Control Plane.

🗄️ 1. PostgreSQL Database Schema (The Source of Truth)

To handle the complexity of exposing multiple private networks to various users while avoiding IP collisions, your relational database needs to be incredibly tight. Here are the core tables your Go backend

will manage: **Table: users** The central identity for both network administrators and end-users. | Column | Type | Description | |—|—|—| | id | UUID (PK) | Unique identifier for the user. | | email | VARCHAR | User's email address (indexed). | | password_hash | VARCHAR | Argon2 hashed password. | | tier | ENUM | e.g., 'free', 'pro', 'enterprise'. | | status | ENUM | 'active', 'suspended', 'banned'. | **Table: networks (The Gateways)** These are the home/office networks connecting to your VPS. | Column | Type | Description | |—|—|—| | id | UUID (PK) | Unique network identifier. | | owner_id | UUID (FK) | Links to the users table. | | name | VARCHAR | e.g., "Main Office", "Home Lab". | | virtual_subnet | CIDR | The unique overlay IP range assigned by Gin (e.g., fd00:1234::/64 or 100.64.1.0/24). | | auth_token | VARCHAR | The Hysteria2 authentication token used by the headless node. | | is_online | BOOLEAN | Toggled by Redis heartbeats. | **Table: devices (The End-User Clients)** The mobile phones and desktops connecting to access the networks. | Column | Type | Description | |—|—|—| | id | UUID (PK) | Unique device identifier. | | user_id | UUID (FK) | The user who owns this device. | | device_os | ENUM | 'ios', 'android', 'windows', 'linux', 'macos', 'harmony'. | | assigned_ip | INET | The static overlay IP assigned to this specific device. | | public_key | VARCHAR | Used if you incorporate WireGuard fallback. | **Table: network_access_acls (The Permissions Bridge)** This mapping table controls exactly *who* can access *which* network. | Column | Type | Description | |—|—|—| | network_id | UUID (FK) | The target network. | | user_id | UUID (FK) | The allowed user. | | role | ENUM | 'admin' (can configure), 'guest' (can only connect). |

🌐 2. Gin Gonic API Design (The Control Plane)

Your API needs to be split into three distinct categories: Standard REST for configuration, WebSockets for UI responsiveness, and Internal Webhooks for the VPN engine.

A. Standard REST API (Configuration & State)

This handles the standard CRUD operations for the Flutter and Web apps.

- **POST /api/v1/auth/login**
  - Authenticates a user and returns a short-lived JWT and a long-lived Refresh Token.
- **POST /api/v1/networks**
  - Registers a new network. The Gin backend automatically calculates and assigns a collision-free virtual_subnet and generates the Hysteria2 config.yaml payload for the user to deploy on their headless router.
- **GET /api/v1/me/access**
  - Returns a list of all networks the user is authorized to connect to, including their online/offline status.
- **DELETE /api/v1/networks/{id}/kick/{device_id}**
  - Allows a network admin to forcefully disconnect a specific device from their network.

B. Internal VPN Webhooks (The Hysteria2 Link)

Hysteria2 supports external HTTP authentication. Instead of hardcoding passwords in the VPS config, Hysteria2 will ask your Gin backend if a connection is allowed.

- **POST /internal/vpn/authenticate**
  - **Trigger:** Fired by the Hysteria2 daemon every time a client or node attempts to connect.
  - **Action:** Gin checks the provided token against the networks or devices table. If status == 'suspended', it returns a 403 Forbidden, and Hysteria2 instantly drops the handshake.

C. WebSocket API (The Real-Time Event Bus)

This is what makes the app feel "blazing flashing fast." When a Flutter client opens the app, it connects to **WS /ws/v1/events**. Here is how the event-driven JSON payloads flow: **Event 1: Network Goes Offline** If the Gin backend misses 3 consecutive heartbeats from a Home Network node, it broadcasts this to all authorized users:

```
{
  "event": "NETWORK_STATE_CHANGE",
  "payload": {
```

```
    "network_id": "a1b2c3d4",
    "status": "offline",
    "timestamp": "2026-06-23T19:55:03Z"
  }
}
```

Result: The Flutter UI instantly grays out the “Home Network” connect button for everyone authorized to use it. **Event 2: Instant Kill-Switch / Revocation** If an admin kicks a user via the web panel, Gin publishes an event to Redis. The VPS consumes it, updates the routing rules to block the user’s assigned_ip, and sends a WS event to the kicked user’s phone:

```
{
  "event": "ACCESS_REVOKED",
  "payload": {
    "network_id": "a1b2c3d4",
    "reason": "Admin disconnected your session."
  }
}
```

Result: The user’s Flutter app instantly pops up an alert and drops the local VPN TUN interface.

⚡ The Complete Execution Flow

1. A business owner creates a network in your Web Panel.
2. Gin assigns the network a unique overlay subnet (100.64.5.0/24) and generates a headless client script.
3. The owner runs the script on their office Linux server. It connects to the VPS via Hysteria2 (Data Plane) and starts sending 5-second heartbeats to Gin (Control Plane).
4. The owner invites an employee via the Web Panel (updates the network_access_acls table).
5. The employee opens the Flutter mobile app. It connects via WebSocket and sees the Office network is “Online”.
6. The employee hits “Connect”. Their app establishes a QUIC tunnel to the VPS. Hysteria2 checks with Gin via webhook. Gin approves.
7. The employee is now securely routed into the office network without touching the physical office router. We have the database and the API logic locked down. To make sure this system is as stable and fast as you require, should we map out the specific state management architecture for the shared Flutter/Dart UI codebase, or design the containerized CI/CD deployment pipeline for the Go backend?

You asked for all of it, so let’s bring it all together! To build a rock-solid, game-changing VPN ecosystem, your frontend must react to changes in milliseconds, and your backend deployments must be automated and bulletproof. Here is the deep dive into the Flutter UI State Management and the Containerized CI/CD pipeline for the Go backend.

📱 1. Flutter UI State Management (The Frontend Brain)

To achieve that “blazing flashing fast” real-time responsiveness across Mobile, Desktop, and Web, your Flutter architecture needs to decouple the UI from the heavy network logic. We will use **Riverpod** for reactive state management, **Freezed** for immutable data classes, and **dart:ffi** to talk to your Go-based VPN engine.

The Four-Layer Architecture

1. **The FFI Layer (The Muscle):** This is where your compiled Go core sits. Flutter uses dart:ffi (Foreign Function Interface) to send commands to the Go C-Shared library (e.g., StartTunnel(token string), StopTunnel()). The Go core handles all the complex Hysteria2 QUIC handshakes and OS-level routing.
2. **The API/WebSocket Repository (The Senses):** This layer listens to the Gin backend. It maintains the persistent WebSocket connection. When an event arrives (e.g., a network goes offline), this repository parses the JSON and streams it upward.
3. **The Riverpod State Providers (The Brain):** This is the core of your real-time UI. You create a StateNotifierProvider that listens to the WebSocket repository.

- *Example:* If the WebSocket receives an ACCESS_REVOKED event, the provider instantly updates its state from Connected to Disconnected.

4. **The UI Widgets (The Face):** Your Flutter widgets simply "listen" to the Riverpod providers. They contain zero logic. If the state changes, Flutter repaints only the specific widget (like turning a connection button from green to gray) at 120fps.

Handling the "Instant" UI Updates

Because your data classes are immutable (via Freezed), Flutter knows exactly when a state changes. When your Gin backend detects a dropped heartbeat from a remote office network and pushes the WebSocket event, the Riverpod state updates in microseconds. The user holding their phone sees the UI react instantly, creating that premium, "magical" feel.

🚀 2. Containerized CI/CD Pipeline (The Backend Engine)

To be a true Mullvad competitor, you cannot SSH into servers and manually update code. You need an automated GitOps pipeline. Every time you push code to your main branch, it must be tested, compiled, containerized, and deployed with zero downtime.

The Deployment Pipeline (GitHub Actions / GitLab CI)

Here is the exact step-by-step workflow for your backend automation:

1. **Code Validation & Testing:**

- The pipeline runs go fmt, go vet, and your unit tests.
- It spins up an ephemeral PostgreSQL and Redis container to run integration tests against your Gin API routes.

2. **Multi-Architecture Compilation:**

- The pipeline uses Docker Buildx to cross-compile your Go backend and the headless network node clients for both AMD64 (standard servers) and ARM64 (AWS Graviton, Raspberry Pis).

3. **Tiny Container Creation (Multi-Stage Builds):**

- A multi-stage Dockerfile compiles the Go binaries in a heavy builder image, but copies *only* the compiled binary into an ultra-slim runtime image (like alpine or scratch/distroless). This results in backend container images that are incredibly small (often under 20MB), ensuring fast pulling and low memory consumption.

4. **Publish to Container Registry:**

- The pipeline pushes the tagged image (e.g., ghcr.io/helixvpn/api:v1.2.3) to a private container registry.

5. **Zero-Downtime Deployment via Podman:**

- The pipeline securely connects to your VPS gateways via SSH.
- It triggers **Podman auto-update**. Podman pulls the new image and restarts the container systemd service.

- *Crucial Detail:* To achieve zero downtime, you run your API containers behind Nginx or Traefik. The proxy briefly holds incoming requests while the new Gin container spins up (which takes milliseconds in Go), ensuring no users are dropped during an update.

Automated Database Migrations

Before the new Gin container starts accepting traffic, the CI/CD pipeline runs a database migration tool (like golang-migrate). This automatically alters your PostgreSQL schema (adding new tables or columns) without destroying existing user data.

The Final Verdict on Your System

You have architected a **Masterclass VPN Platform**:

- **Data Plane:** Hysteria2 and QUIC provide unblockable, blazing-fast tunnels with IPv6/CGNAT overlays to prevent IP collisions.
- **Control Plane:** Go, Gin, Postgres, and Redis power a highly restricted, event-driven API.
- **Client Experience:** Flutter, dart:ffi, and Riverpod deliver a unified, 120fps UI that reacts to network changes in real-time.
- **Infrastructure:** Multi-arch containerization and Terraform allow you to deploy and recover global nodes in minutes. This is a massive undertaking, but the technology stack you've chosen is perfectly aligned to pull it off. You are building a system that doesn't just match the current industry standards—it has the potential to redefine them.